

CISC 1115 (TY8) Exam 1 Review Sheet

NOTE: This review sheet covers most (but not all) topics for your upcoming exam. It is here to clear up any confusion you may have about a particular concept; however, please keep in mind that we cannot and will not provide you with all the answers. Programming is a hands-on skill; please take it upon yourself to understand these concepts and use them. Please work on the sample exercises provided; the only way to get better at writing code is by **actually writing code**.

Topics

- Chapter 2: Variables and operators
- Chapter 3: Input and output
- Chapter 5: Conditionals and Logic
- Chapter 7: Loops

Chapter 2: Variables and Operators

Identifiers and Keywords

- Keywords are predefined, reserved words used in Java programming with special meanings to the compiler.
 - **For example: int, char, double, new, return, static, etc.**
- You cannot use keywords like int, for, class, etc. as variable names (or identifiers) as they are part of the Java programming language syntax.
- Identifiers are the names given to variables, classes, methods, etc.
 - **For example: n, num, number, score, etc.**

Types of Programming Errors

- **Syntax/Compiler Error:** An error that prevents the program from compiling successfully.
- **Run-time Error:** An error that occurs while the program is running.
- **Logical Error:** An error where the program runs, but does not produce the intended result.

When debugging code, you should be able to identify the error type, determine what causes it, and correct the code.

Arithmetic Expressions

Operator	Name	Description	Example
+	Addition	Adds together two values	<code>x + y</code>
-	Subtraction	Subtracts one value from another	<code>x - y</code>
*	Multiplication	Multiplies two values	<code>x * y</code>
/	Division	Divides one value by another	<code>x / y</code>
%	Modulus	Returns the division remainder	<code>x % y</code>
++	Increment	Increases the value of a variable by 1	<code>++x</code>
--	Decrement	Decreases the value of a variable by 1	<code>--x</code>

- Pre-increment (`++x`), i.e. `System.out.println(++x);` $\Rightarrow \Rightarrow \Rightarrow$ result: new value of `x`
- Post-increment (`x++`), i.e. `System.out.println(x++);` $\Rightarrow \Rightarrow \Rightarrow$ result: original value of `x`, store new value for next action
- `(=)`: assignment vs `(==)`: equivalence

Arithmetic Precedence

Java evaluates arithmetic expressions according to the following order of precedence:

Precedence	Operator(s)	Description	Example
1 (Highest)	<code>()</code>	Parentheses	<code>(x + y) * z</code>
2	<code>++ --</code>	Increment / Decrement	<code>x++, --y</code>
3	<code>* / %</code>	Multiplication, Division, Modulus	<code>x * y, x / y, x % y</code>
4 (Lowest)	<code>+ -</code>	Addition, Subtraction	<code>x + y, x - y</code>

Important:

- Expressions inside **parentheses** are evaluated first.
- Operators at the **same precedence level** are generally evaluated from **left to right**.
- Parentheses can be used to change the order in which an expression is evaluated.

Example:

```
int result = 2 + 3 * 4;    // 14
int result2 = (2 + 3) * 4; // 20
```

Java.lang.Math class methods

- Math Class methods are built-in methods that make numeric operations like square, square root, cube, cube root, exponential, and trigonometric operations easier.
- Math class methods include (but are not limited to):
 - **abs():** java.lang.Math.abs(datatype arg) method returns the absolute value of any type of argument passed. This method can handle all data types.
 - **Parameters:**
arg: the argument whose absolute value we need
 - **Returns:**
the absolute value of the passed argument
 - **pow():** java.lang.Math.pow(double b, double e) method returns the value as b^e
 - **Parameters:**
b: base
e: exponent
 - **Returns:**
value as $\text{base}^{\text{exponent}}$
 - **sqrt():** java.lang.Math.sqrt(double a) method returns the correctly rounded positive square root of a double value
 - **Parameters:**
a - a value
 - **Returns:**
the positive square root of a. If the argument is NaN or less than zero, the result is NaN
 - **random():** java.lang.Math.random() method returns a pseudorandom double type number greater than or equal to 0.0 and less than 1.0
 - **Returns:**
a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.
 - **To generate a double value within a specific range:**
 $\text{Math.random()} * (\text{max} - \text{min}) + \text{min}$

Where min is inclusive, and max is exclusive

(i.e., to generate a number between 1 and 10, set min to 1 and max to 11)

- **See the Java API for the full list of Math class methods:**
<https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/lang/Math.html>.

Chapter 3: Input and Output

Writing to the Screen (System.out)

- System.out represents the Standard Output Stream. That means that if we want to print any statement on the console, we should use the following statement:

System.out.print();

- There are three methods we use to print statements:
 - **print(String s)**: used to print on the console. It accepts a string as a parameter (we can concatenate parts to the string if necessary, i.e., "Hello" + " World", etc.; however, it is typical convention to include all string literals in the same pair of quotation marks if they are consecutive). After printing the statement, the cursor remains on the same line.
 - **println(String s)**: upgraded version of the print() method. It is also used to display text on the console. After printing the statement, it moves the cursor to the start of the next line. This is the main difference between println() and print().
 - **printf(String format, datatype args)**: It accepts two parameters:
 - **format**: It is a formatted String that uses placeholders with specific conversion characters to refer to the arguments that will be replaced when printing
 - **args**: It is an argument referenced by the format specifiers. If the number of arguments is more than the format specifiers, the other arguments are ignored. The number of arguments may be zero.
 - Conversion characters are only valid for certain data types. Here are some common ones:
 - **%s formats strings (String)**
 - **%d formats decimal integers (int)**
 - **%f formats floating-point numbers (double)**
 - **Optional Modifiers**:
 - **The [flags] define standard ways to modify the output and are most common for formatting integers and floating-point numbers.**
 - **The [width] specifies the field width for outputting the argument. It represents the minimum number of characters written to the output.**

- The **[.precision]** specifies the number of digits of precision when outputting floating-point values. You can also use it to define the length of a substring to extract from a **String**.
- **%[flags][width][.precision]conversion-character** (i.e., **%8.2f** would print a double value with two decimal places and a minimum field width of 8 characters, adding spaces to the left if necessary.)
- **printf()** throws **NullPointerException** if the format is null, also throws the **IllegalFormatException** if a format string contains illegal syntax, and throws an **IllegalFormatConversionException** if the placeholder and corresponding argument are not of the same type.

Read Interactively from the Keyboard (**System.in**)

- All reading done in Java uses the **Scanner** class. Using this class, we can create an object to read input from the standard input channel **System.in** (the keyboard)
- To use the **Scanner** class, it is necessary to import the class **Scanner** from the library **java.util** by including the following line at the top of the program:

```
import java.util.Scanner;
```

- We can declare a **Scanner** object by saying:

```
Scanner sc = new Scanner (System.in);
```

- where:
 - **sc** is the name of the **Scanner** object (this can be changed and is entirely up to the person who writes the code)
 - and **System.in** connects our object to standard input (the keyboard)
- The **Scanner** class has multiple built-in methods that we can access to help us read data; for example, **we can use `sc.nextLine()` to read in a line of **String(s)** (spaces included)**
- Other methods include:
 - **`nextBoolean()`: reads a boolean value from the user**
 - **`nextByte()`: reads a byte value from the user**
 - **`nextDouble()`: reads a double value from the user**
 - **`nextFloat()`: reads a float value from the user**
 - **`nextInt()`: reads an int value from the user**

- **next():** reads a complete token (String) from the user. A complete token is preceded and followed by input that matches the delimiter pattern; in most instances, this is a space.
- **Sample exercises:**
 1. **Write a Java program that prompts the user for their first and last name and then prints out a message: "Hello, [FIRST NAME, LAST NAME]!"**

Sample Data (Input/Output):

**Enter your name: Jane Doe
Hello, Jane Doe!**

Reading and Writing Using Files

- As mentioned, we use a Scanner to read data, whether it comes from the keyboard or a file. **To read from a file, we only need to make one or two modifications from what we learned from reading from the keyboard.**
- First, since we are using files, we need to allow our program(s) to use them by including:

import java.io.File; //import the File class

- Now we can add a **throws Exception** declaration to main and create the File object that we will be reading from:

File file= new File("[filename].txt");

- And create a Scanner as we have previously done, only changing our reference from the keyboard to the corresponding file object:

Scanner sc = new Scanner(file);

- Remember to **create the .txt file you are trying to read from, and make sure it is in the correct directory, or specify the file's path when creating the File object;** otherwise, you'll get a FileNotFoundException when you run your code.
- Writing to a file is slightly less work because we don't have to manually create the .txt file before running the program; the program creates it for us.
- Like before, we need to import the PrintWriter class to use it:

import java.io.PrintWriter;

- We will use the `PrintWriter` class to create our file:

```
PrintWriter pw = new PrintWriter("[filename].txt");
```

- **Make sure the filename you are trying to create does not already exist in the directory you are writing to; otherwise, you will overwrite the file and lose your original data.**
- Now we can use the `PrintWriter` object to write, the same way that we wrote to the screen, only replacing `System.out` with our `PrintWriter` object, i.e. **`pw.print()`, `pw.println()`, `pw.printf()`**
- **Try modifying the programs that you may have previously done so that they write to both the screen AND to a file.**

Closing Files

- When you are finished reading from or writing to a file, close the corresponding `Scanner` and/or `PrintWriter`:

```
sc.close();  
pw.close();
```

- Closing a `PrintWriter` ensures that it writes any remaining output to the file.

Chapter 5: Conditionals and Logic

If and if-else statements

- An if statement specifies a statement(s) to be executed only if a particular boolean expression is true.
- An if statement can be followed by an optional else statement, which executes when the boolean expression is false.
- An if statement can be followed by an else if statement to specify a new condition if the first condition is false. If you choose to include an else if statement, it **MUST COME BEFORE** the else statement.

- Syntax:

```
○ if(condition1) {  
    //statement(s) to be executed if condition1 is true  
}  
  else if(condition2) {  
    //statement(s) to be executed if the condition1 is false and  
    //condition2 is true  
}  
  else {  
    //statement(s) to be executed if the condition1 is false and  
    //condition2 is false  
}
```

- Sample exercises:

1. Write a Java program that prompts the user for a number and prints whether it is even or odd.

Sample Data (Input/Output):

Enter number: 104
104 is an even number.

2. Write a Java program that prompts the user for three numbers and prints out the largest of the three.

Sample Data (Input/Output):

Enter three numbers: 78 25 87
The largest number is: 87

Switch Statements

- A switch statement lets a program select between several possible actions based on the value of a variable or expression.

Traditional Switch Statement

```
switch (choice) {  
    case 1:  
        System.out.println("Option 1");  
        break;  
    case 2:  
        System.out.println("Option 2");  
        break;  
    case 3:  
        System.out.println("Option 3");  
        break;  
    default:  
        System.out.println("Invalid option");  
}
```

- The switch expression compares its value with each case, and when it finds a match, it executes the statements for that case.
- **break** exits the switch statement and prevents execution from continuing into the next case.
- **default** executes when none of the cases match.

Rule Switch/Arrow Notation (->)

- Java also allows cases to use the **arrow (->) syntax**:

```
switch (choice) {  
    case 1 -> System.out.println("Option 1");  
    case 2 -> System.out.println("Option 2");  
    default -> System.out.println("Invalid option");  
}
```

- The code to the right of -> executes when that case matches, so a break statement is **not needed** with the arrow syntax.
- Multiple case values can share the same rule:

```
switch (choice) {  
    case 1, 2 -> System.out.println("Option A");  
    case 3, 4 -> System.out.println("Option B");  
    default -> System.out.println("Invalid option");  
}
```

- **Sample Exercise:**

1. Write a Java program that prompts the user to enter a number from 1–3 and uses a switch statement to print:

- 1 → "Red"
- 2 → "Green"
- 3 → "Blue"
- Any other value → "Invalid choice"

Nested if-else statements

- Nested if statements mean an if statement inside an if statement

- Syntax:

- if (condition1) {
 //statement(s) to be executed if condition1 is true
 if (condition2) {
 //statement(s) to be executed if condition2 is true
 }
}

- **Sample exercises:**

1. Write a Java program that prompts the user for a year and prints whether that year is a leap year or not. (*Note: Leap Years are any year that can be evenly divided by 4. A year that is evenly divisible by 100 is a leap year only if it is also evenly divisible by 400. For example, 1700, 1800, and 1900 are not leap years, but 1600 and 2000 are.*)

Sample Data (Input/Output):

Enter year: 2016

2016 is a leap year.

Logical and Relational Operators

- Boolean expressions (evaluate to true/false):
 - Less than: **a < b**
 - Less than or equal to: **a <= b**
 - Greater than: **a > b**
 - Greater than or equal to: **a >= b**
 - Equal to: **a == b**
 - Not Equal to: **a != b**

Integer and Logical Values

- We can include integers in our boolean expressions and determine whether or not they are true or false. For instance, if we were to print out:
 - `System.out.print(6 > 5)`, we would see a result value of 'true'
 - while `System.out.print(6 < 5)` or `System.out.print(6 == 5)` would result in a value of 'false' for obvious reasons
- Logical operators allow us to combine or modify Boolean expressions.

Operator	Name	Description	Example
&&	AND	True only when both conditions are true	<code>x > 5 && x < 10</code>
	OR	True when at least one condition is true	<code>x < 5 x > 10</code>
!	NOT	Reverses the value of a Boolean expression	<code>!(x > 5)</code>

- For example:
 - `int x = 8;`
 - `System.out.println(x > 5 && x < 10); // true`
 - `System.out.println(x < 5 || x > 10); // false`
 - `System.out.println(!(x > 5)); // false`

Chapter 7: Loops

While Loops

- allows code to be executed repeatedly based on a given Boolean condition (T/F). Needs two parts: **condition/test expression** and **increment/decrement/update expression**
- Syntax:
 - ```
int i = 1;
while(i <= 10) {
 //statement(s) to be executed while the condition is true
 i++;
}
```
  - where:
    - `i <= 10` is a boolean expression. **If the condition evaluates to true, we will execute the body of the loop and go to the update expression.** Otherwise, we will exit from the while loop without reaching the statements inside the body.
    - And `i++` increments the variable `i`
- How many times does this loop run? What would happen if you forgot to include `i++`;  
Try writing code that uses a while loop like this.

### Sentinel-Controlled Loops

- A **sentinel value** is a special value that indicates when input should stop.
- A sentinel-controlled loop continues reading and processing data until the sentinel value is entered.
- For example:
  - ```
System.out.print("Enter a number (0 to stop): ");
int number = stdin.nextInt();

while (number != 0) {
    System.out.println("You entered: " + number);

    System.out.print("Enter another number (0 to stop): ");
    number = stdin.nextInt();
}
```
- Notice that input is read **before the loop** and again **inside the loop** so that the condition can eventually become false.

Reading Until End-of-File (EOF)

- When reading from a file, you may not always know how many values are in the file. You can use a while loop to keep reading as long as more input is available.
- Scanner methods such as **hasNextInt()** and **hasNextDouble()** can be used to check whether another value is available before reading it.

- Example:

```
while (sc.hasNextInt()) {  
  
    int number = sc.nextInt();  
  
    System.out.println(number);  
  
}
```

- The loop stops when there are no more integers to read from the file.

Do-While Loops

- A do-while loop is similar to a while loop, except it checks the condition after the loop body executes.
- Because of this, a do-while loop will always execute at least once.

- Syntax:

```
int i = 1;  
  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 10);
```

- Key Difference:
 - **while** checks the condition **before executing the loop body**.
 - **do-while** checks the condition **after executing the loop body**.
- Therefore, a while loop may execute zero times, while a do-while loop executes at least once

- **Sample exercises:**

1. Write a Java program that prompts the user for age, prints “Sorry, you are ineligible to vote.”, and increments the age until the user is eligible to vote, then prints “Congratulations, you are eligible to vote!” once they are 18+.

Sample Data (Input/Output):

Enter your age: 16

Sorry, you are ineligible to vote.

Sorry, you are ineligible to vote.

Congratulations, you are eligible to vote!

2. Write a Java program that prompts the user to input an integer and then outputs the number with the digits reversed. For example, if the input is 12345, the output should be 54321.

Sample Data (Input/Output):

Enter a number: 12345

Reverse of 12345 is: 54321

For Loops

- Unlike a while loop, a for statement has the **initialization, condition, and increment/decrement** in one line.
- Syntax:
 - ```
for (int i = 1; i <= 10; i++) {
 //statement(s) to be executed while the condition is true
}
```
  - where:
    - `int i = 1;` is the initialization
    - `i <= 10;` is a boolean condition
    - `And i++` increments the variable `i`
- How many times does this loop run? Try writing code that uses a for loop like this.

## Using a Header Value

- Sometimes a file contains a header value that tells us how many data values or records follow.
- Since the number of repetitions is known, you can use a for loop to process the data.
- Example file:

4  
10  
20  
30  
40

- Example:

```
int count = sc.nextInt();

for (int i = 0; i < count; i++) {
 int number = sc.nextInt();
 System.out.println(number);
}
```

- The header value is read before the loop and determines how many times the loop executes.

- Sample exercises:

1. Write a program to print numbers from 10 to 100 in increments of 10.

Sample Data (Output only):

10 20 30 40 50 60 70 80 90 100

2. Write a program that prompts the user to input a positive integer. It should then print the multiplication table of that number. (0 - 12)

Sample Data (Input/Output):

Enter a number: 5  
Multiplication Table of 5  
5 x 0 = 0  
5 x 1 = 5  
.... ..  
5 x 12 = 60

## Choosing the Appropriate Loop

| Situation                            | Typical Loop | How Repetition Ends              |
|--------------------------------------|--------------|----------------------------------|
| Number of repetitions is known       | for          | Counter reaches its limit        |
| File provides a header/count         | for          | Counter reaches the header value |
| Number of repetitions is unknown     | while        | Condition becomes false          |
| Special value indicates the end      | while        | Sentinel value is entered        |
| Reading all values from a file       | while        | End-of-file is reached           |
| Loop body must execute at least once | do-while     | Condition becomes false          |

## More Complex Loops

- **Nested loops:** if a loop exists inside the body of another loop, it's called a nested loop.
- **Loops with more than one condition:** a condition is a Boolean expression and can have more than one part to be considered.
- For example, to vote, you need to be 18+ AND an American citizen. In this case, a person is eligible to vote only when both requirements are true. It works the same with loop conditions.
  - When using AND (&&), both parts of the condition must be true to evaluate to true
  - When using OR (||), only one part needs to be true to evaluate to true
- **Sample exercises:**
  1. **Write a Java program that prints out each day of the week for a month (approximately 4 weeks).**

### Sample Data (Output):

Week: 1

Day: 1

Day: 2

Day: 3

```

.....
Week: 2
 Day: 1
 Day: 2
 Day: 3

.....

```

2. Write a Java program that prompts the user for a number of rows and then prints out a half pyramid of asterisks (‘\*’) with the same number of rows.

Sample Data (Input/Output):

Enter the number of rows: 5

```

*
* *
* * *
* * * *
* * * * *

```

3. Write a Java program that prompts the user for a number less than 100 and prints out all the even numbers starting from the number that they entered up to and including 100.

Sample Data(Input/Output):

Enter a number: 85

86 88 90 92 94 96 98 100

## Tracing Code

When tracing code, follow the program **one** statement at a time and **keep track of how variable values change**.

- Evaluate expressions and conditions **in the order they are executed**.
- Determine which statements execute.
- **Keep track of any changes to variable values.**
- For loops: keep track of **each iteration** and when the loop condition becomes false.
- Continue through the program using the **updated values**.
- Determine the exact output produced by the code.